INGEGNERIA DEL SOFTWARE

ESERCITAZIONE 3

Gabriele Paveri-Fontana
Mail: gabriele.paveri@polimi.it

21 Ottobre 2026

Esercizio 1a *Polygon* (Classe astratta)

Progettare (UML) e implementare (Java) le classi
Polygon, *Rectangle*, *Square* e *Triangle*.

Specifiche:

- *Polygon* è una classe astratta che definisce:
  - ✓ il metodo astratto *getPerimeter()*;
  - ✓ il metodo statico *printPerimeters(…)*: stampa il perimetro di un array di poligoni;
  - ✓ il metodo *hasGreaterPerimeter(Polygon)*, che restituisce un booleano.
- *Rectangle* e *Triangle* sono sottoclassi di *Polygon*, ognuna con la propria implementazione del metodo *getPerimeter()*.
- Valutare come implementare *Square* ed il suo metodo *getPerimeter()*, ed aggiungere a *Square* il metodo *diagonal()* che restuisce la lunghezza della diagonale.

Esercizio 1b *Polygon* (Interfaccia)

Progettare (UML) e implementare (Java) l'interfaccia *Polygon* e le classi *Rectangle*, *Square* e *Triangle*.

Specifiche:

- *Polygon* è una interfaccia che definisce:
  - ✓ il metodo astratto *getPerimeter()*;
  - ✓ il metodo statico *printPerimeters(…)*: stampa il perimetro di un array di poligoni;
  - ✓ il metodo *hasGreaterPerimeter(Polygon)*, che restituisce un booleano. (!!)
- *Rectangle* e *Triangle* implementano l'interfaccia *Polygon*, ognuna con la propria implementazione del metodo *getPerimeter()*.
- Valutare come implementare *Square* ed il suo metodo *getPerimeter()*, ed aggiungere a *Square* il metodo *diagonal()* che restuisce la lunghezza della diagonale.

Esercizio 1c Compile-time VS Run-time

Sia dato il seguente programma (con import del pacchetto relativo ad 1a). Dovete:

1) Individuare ed eliminare le istruzioni che generano errori a compile-time.
2) Individuare ed eliminare le istruzioni che generano errori a runtime.
3) Stabilire cosa produce in output il programma.

Esercizio 1c Compile-time VS Run-time

```
import PolygonAbstractClass.*
public class Example {
   public static void main(String[] args) {
      Polygon p = new Polygon();                //1
      Polygon r = new Rectangle(1.5, 3);        //2
      Polygon s = new Square(2.5);              //3
      Polygon t = new Triangle(3, 4, 5);        //4
      Triangle tr = t;                          //5
      System.out.println(r.getPerimeter());     //6
      System.out.println(s.getPerimeter());     //7
      System.out.println(t.getPerimeter());     //8
      System.out.println(tr.getPerimeter());    //9
      System.out.println(s.diagonal());         //10
      Square sq1 = (Square) s;                  //11
      System.out.println(sq1.diagonal());       //12
      Square sq2 = (Square) r;                  //13
   }
}
```

Esercizio 1c Compile-time VS Run-time

```
import PolygonAbstractClass.*
public class Example {
   public static void main(String[] args) {
      Polygon p = new Polygon();                  //1
      Polygon r = new Rectangle(1.5, 3);          //2
      Polygon s = new Square(2.5);                //3
      Polygon t = new Triangle(3, 4, 5);          //4
      Triangle tr = t;                            //5
      System.out.println(r.getPerimeter());       //6
      System.out.println(s.getPerimeter());       //7
      System.out.println(t.getPerimeter());       //8
      System.out.println(tr.getPerimeter());      //9
      System.out.println(s.diagonal());           //10
      Square sq1 = (Square) s;                    //11
      System.out.println(sq1.diagonal());         //12
      Square sq2 = (Square) r;                    //13
   }
}
```

Errori a compile-time e run-time

- //1 errore a compile-time (una classe astratta non può essere istanziata).
- //5 errore a compile-time (il tipo statico di *t* è *Polygon*, che non è un sottotipo di *Triangle*).
- //9 eliminando la riga //5, la variabile *tr* risulta non definita, da cui un errore a compile-time.
- //10 errore a compile-time (l'oggetto *s* ha tipo statico *Polygon*, dunque non ha accesso al metodo *diagonal()* di *Square*).
- //13 errore a run-time (*Square* non è sovratipo del tipo dinamico di *r*, che è *Rectangle*).

Esercizio 1c Compile-time VS Run-time

```
import PolygonAbstractClass.*
public class Example {
   public static void main(String[] args) {
      Polygon r = new Rectangle(1.5, 3);        //2
      Polygon s = new Square(2.5);              //3
      Polygon t = new Triangle(3, 4, 5);        //4
      System.out.println(r.getPerimeter());     //6
      System.out.println(s.getPerimeter());     //7
      System.out.println(t.getPerimeter());     //8
      Square sq1 = (Square) s;                  //11
      System.out.println(sq1.diagonal());       //12
   }
}
```

Output del programma

- //6 «9»
- //7 «10»
- //8 «12»
- //11 «3.53»

Esercizio 2 *java.lang.Comparable*

Estendere le classi *Person* e *Student* (lab2.ex6) implementando l'interfaccia *java.lang.Comparable*.

Specifiche:

- *Person* implementa l'interfaccia *Comparable<Person>* controllando l'ordine lessicografico dei cognomi e poi, in caso di omonimia, dei nomi.
- *Student* aggiunge a questo comportamento, in caso di omonimia sia sul nome che sul cognome, il controllo sull'id.

Recap Exceptions

Checked Exceptions

- Controllate a compile-time
- Sottoclassi di *java.lang.Exception*
- Devono essere catturate (*try-catch-finally*) o dichiarate nella signature del metodo (*throws*)
- Usate di solito per situazioni fuori dal controllo del programmatore, da cui il programma deve recuperare
- Esempi: IOException, FileNotFoundException

Unchecked Exceptions

- Controllate a run-time
- Sottoclassi di *java.lang.RuntimeException*
- Non è necessario né catturarle né dichiararle nella signature. Possono comunque essere propagate (*throw*)
- Usate di solito per indicare errori di programmazione, da cui il programma non può recuperare
- Esempi: NullPointerException, IndexOutOfBoundsException

Esercizio 3 *BankAccount*

Progettare (UML) e implementare (Java) un'applicazione che simuli le operazioni di una banca.

Specifiche:

- Classe *BankAccount*: caratterizzata da un identificativo *id* (int) e un bilancio *balance* (double), espone i metodi *deposit(double)* per depositare denaro e *withdraw(double)* per prelevare denaro.
- Eccezione controllata *NegativeAmountException*: per gestire il caso in cui si cerchi di prelevare o depositare una quantità negativa di denaro.
- Eccezione controllata *InsufficientFundsException*: per gestire il caso in cui si cerchi di prelevare più denaro di quanto non si possieda.

Esercizio 4 *SafeStack*

Progettare (UML) e implementare (Java) la classe *SafeStack*, rendendola tipizzabile.

Specifiche:

- *SafeStack* implementa le funzionalità di uno stack, utilizzando un *ArrayList* per immaganizzare i dati.
- Rendere la classe *SafeStack* tipizzabile con i Generics (*SafeStack<T>*).
- *SafeStack<T>* espone il metodo *safePush(T)*, che gestisce correttamente i casi limite dell'inserimento di nuovi elementi (con eccezioni controllate).
- *SafeStack<T>* espone il metodo *safePop()*, che gestisce correttamente i casi limite dell'estrazione di un elemento (con eccezioni controllate).

Teoria Enumerated Class (Enum)

- Il tipo (speciale) *Enum* (tipo enumerato) è fondamentalmente un modo per vincolare una variabile a poter assumere solo un limitato set di valori determinato dallo sviluppatore.
- Ogni *Enum* è una classe (speciale), sottoclasse di *java.lang.Enum*.
- Le istanze di una *Enum* possono valere solo *null* oppure uno dei valori enumerati, e non si possono costruire altre istanze.
- A una *Enum* si possono aggiungere costruttore, metodi e attributi.
  - ✓ Il costruttore è invocato solo quando vengono costruiti i valori enumerati e non può essere chiamato esplicitamente
- Non c'è bisogno di usare *equals* per confrontare due variabili di un tipo enumerato, basta == (è sovrascritto e si comporta come *equals*).
- Per ogni *Enum* il compilatore sintetizza un metodo statico *values()*, che restituisce un array di tutti i possibili valori assumibili dalle variabili.

Teoria Enumerated Class (Enum)

```
public enum Size {
        SMALL, MEDIUM, LARGE
}
```

```
  Size size = Size.MEDIUM;

  for (Size s: Size.values()) {
          System.out.println(s);
  }
```

```
public enum Size {
        SMALL("S"), MEDIUM("M"), LARGE("L");

        private String abbreviation;

        (private) Size(String abbreviation) {
            this.abbreviation = abbreviation;
        }

        public String getAbbreviation() {
            return abbreviation;
        }
}
```

Teoria Enumerated Class (Enum)

```
public enum Planet {
  MERCURY (3.303e+23, 2.4397e6), VENUS (4.869e+24, 6.0518e6), EARTH (5.976e+24, 6.37814e6),
  MARS (6.421e+23, 3.3972e6), JUPITER (1.9e+27, 7.1492e7), SATURN (5.688e+26, 6.0268e7),
  URANUS (8.686e+25, 2.5559e7), NEPTUNE (1.024e+26, 2.4746e7), PLUTO (1.27e+22, 1.137e6);

  private final double mass; // in kilograms
  private final double radius; // in meters

  Planet(double mass, double radius) {
    this.mass = mass; this.radius = radius;
  }

  public double mass() {return mass;}
  public double radius() {return radius;}

  // universal gravitational constant (m^3 kg^-1 s^-2)
  public static final double G = 6.67300E-11;

  public double surfaceGravity() {return G * mass / (radius * radius);}
  public double surfaceWeight(double otherMass) {return otherMass * surfaceGravity();}
}
```

```
public static void main(String[] args) {
  double earthWeight = 70;
  double mass = earthWeight / Planet.EARTH.surfaceGravity();

  for (Planet p : Planet.values())
    System.out.println("Your weight on " + p +
                       " is " + p.surfaceWeight(mass));
}
```

Esercizio 5 Menu per *SafeStack*

Progettare un nuovo *main* per la classe *SafeStack* (lab3.ex4).

Specifiche:

- Definire una *Enum* con i valori *CREATE, PUSH, POP, PRINT, QUIT*.
- In loop, il nuovo *main* stampa le opzioni possibili e legge da input la scelta dell'utente (gestendo tramite eccezioni gli input non accettabili), per poi operare di conseguenza.
- Il programma termina quando viene scelta l'opzione *QUIT*.